

Introduction

1/ Logique formelle : définition

- Système de raisonnement basé sur la **forme des énoncés**, indépendamment de leur contenu.
- Étudie les **règles de déduction** à l'aide de méthodes formelles pour garantir la **validité** des raisonnements.

2/ Intérêt

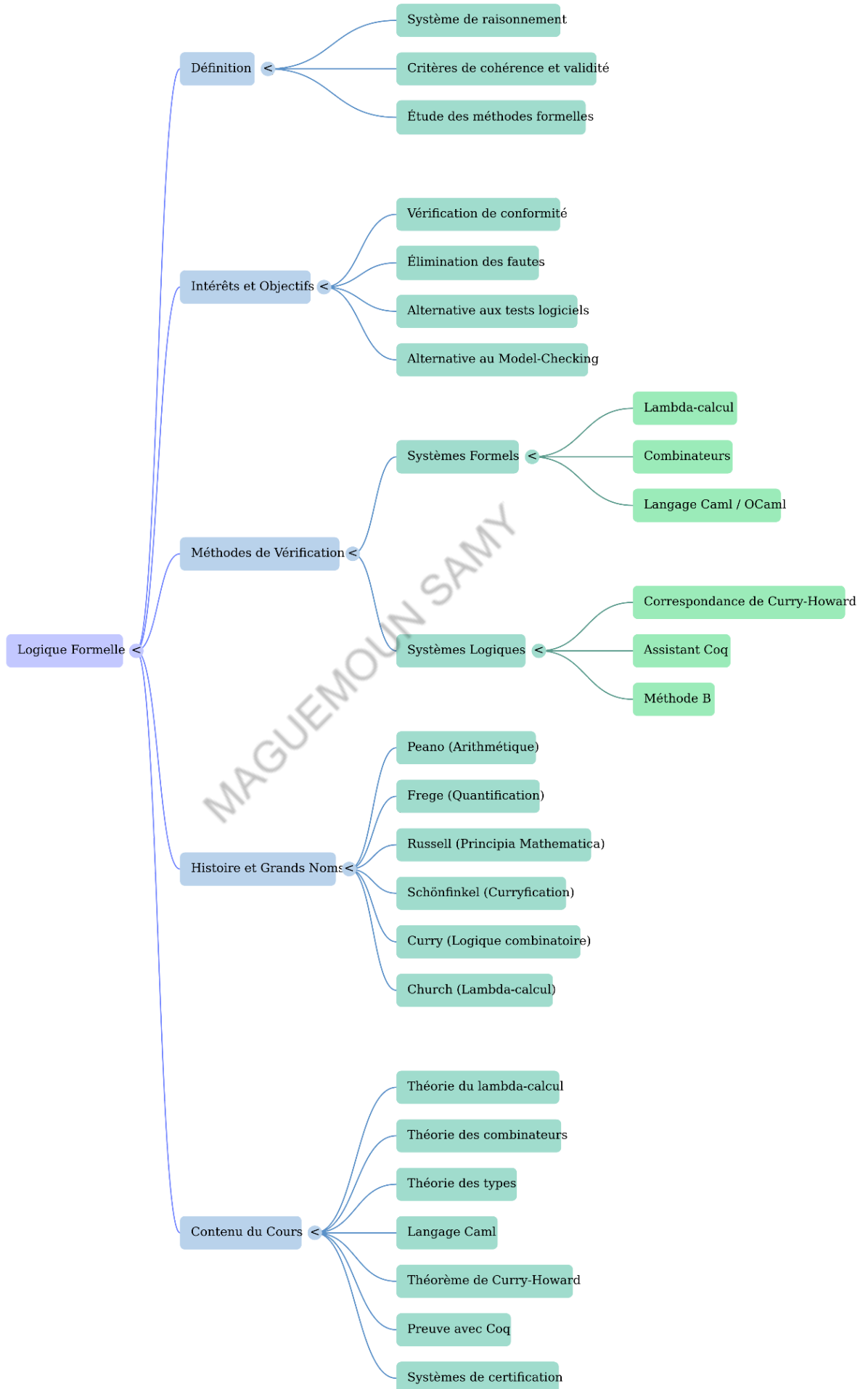
- Permet de **vérifier qu'un programme respecte sa spécification**.
- Aide à **éviter les bugs avant le déploiement**.
- Plus fiable que :
 - les **tests logiciels** (non exhaustifs, coûteux)
 - le **model-checking** (difficile pour systèmes complexes).

3/ Comment faire

- Utiliser des **systèmes formels** pour décrire les programmes :
 - lambda-calcul, combinateurs, lambda-calcul typé, OCaml.
- Relier ces systèmes à la logique pour prouver :
 - correspondance de Curry-Howard (ex : Coq)
 - méthode B.

4/ Quelques grands noms

- **Peano (1889)** : arithmétique formelle.
- **Frege (1891)** : logique et quantification.
- **Russell (1910)** : Principia Mathematica.
- **Schönfinkel (1920)** : combinateurs.
- **Curry (1926)** : logique combinatoire.
- **Church (1936)** : lambda-calcul (avec Turing, calculabilité).



Chapitre 1 : Le lambda-calcul

1/ Introduction

Qu'est ce que le λ -calcul

Le λ -calcul est un système formel développé par Church dans les années 1930, qui permet d'exprimer tous les calculs uniquement à l'aide de fonctions. Il constitue un modèle de calcul universel, équivalent aux machines de Turing, et sert de base aux langages de programmation fonctionnels. On le considère souvent comme le plus petit langage de programmation existant.

λ -calcul (illustration)

- Les fonctions sont écrites avec **une seule entrée** (curryfication).
- Exemples :
 - Identité : $\lambda x.x$
 - Différence : $\lambda x.(\lambda y. - x y)$
- Les opérations (comme “-”) sont aussi vues comme des **fonctions**.
- En λ -calcul, **tout est représenté par des fonctions** (nombres, listes, etc.)

À propos des fonctions

- **Mathématiques** :
Deux fonctions sont égales si elles donnent le **même résultat** pour chaque entrée → vue **extensionnelle**.
- **Informatique** :
On considère aussi la **façon de calculer** (temps, mémoire, formule) → vue **intensionnelle**.
- **λ -calcul** :
Adopte une vision **intensionnelle** des fonctions (basée sur leur définition).

λ -calcul dans les langages

- Le λ -calcul est à la base des **langages fonctionnels** comme OCaml, Haskell, Lisp et Python.
- Même les langages **impératifs** ont intégré ces concepts :
 - PHP (closures, lambdas)
 - C++ (lambdas)
 - Java (lambda-expressions)
 - JavaScript (arrow functions)

2/ Définition formelle du λ -calcul :

- Soit un **ensemble infini de variables** $\mathcal{V}(x, y, z, \dots)$.
- Soit un **alphabet** $A = \{\text{éléments de } \mathcal{V}, (,), \lambda, .\}$.
- L'ensemble des **λ -termes** Λ est le plus petit ensemble tel que :
 1. **Variable** : si $x \in \mathcal{V}$ alors $x \in \Lambda$
 2. **Application** : si $M, N \in \Lambda$ alors $(M N) \in \Lambda$

3. **Abstraction** : si $x \in \mathcal{V}$ et $M \in \Lambda$ alors $(\lambda x . M) \in \Lambda$

Exemples de λ -termes :

- **Variable** : x
- **Application** : $(M N)$ (appliquer M à N , M = fonction et N = argument)
 - Exemple : si M est une fonction carré ($x \mapsto x^2$) et $N = 3$, alors $(M N) = 3^2 = 9$
- **Abstraction** : $(\lambda x . M)$ (fonction prenant x et retournant M , x = variable liée)
 - Exemple : $(\lambda x . x + 2)$ prend x et retourne $x + 2$. Si on applique cette fonction à $N = 3$: $(\lambda x . x + 2) 3 = 3 + 2 = 5$
- **Fonction composition** : $(\lambda g . (\lambda f . (\lambda x . g (f x)))) \rightarrow$ correspond à $g(f(x))$
- **Fonction complexe** : $(\lambda x . (\lambda y . (\lambda z . ((x z) (y z)))))$

Remarque :

- Dans une **application** $(M N)$, M est toujours la fonction et N son argument.
- Lorsqu'on **compose deux fonctions** g et f , on obtient $g(f(x))$.

3/Conventions pratiques :

1. **Parenthèses externes** : on peut les omettre.
 - $(M N)$ devient $M N$.
2. **Application associée à gauche** : $M N P$ signifie $(M N) P$.
3. **Corps d'une λ -abstraction** : s'étend le plus à droite possible.
 - $\lambda x . M N = \lambda x . (M N)$, pas $(\lambda x . M) N$.
4. **Grouper des λ -abstractions** :
 - $\lambda x y z . M = \lambda x . \lambda y . \lambda z . M$.
5. **Notation** : variables en **minuscule**, λ -termes en **majuscule**.

Exemples :

- Terme initial : $(\lambda x . (\lambda y . (\lambda z . ((x z) (y z)))))$
 1. Supprimer parenthèses externes : $\lambda x . (\lambda y . (\lambda z . ((x z) (y z))))$
 2. Application associative à gauche : $\lambda x . (\lambda y . (\lambda z . (x z (y z))))$
 3. Parenthèses autour des abstractions : $\lambda x . \lambda y . \lambda z . x z (y z)$
 4. Grouper les abstractions : $\lambda x y z . x z (y z)$
- Autre exemple : $\lambda x y . x y \lambda z . y z x$
 1. Dégroupier : $\lambda x . \lambda y . x y \lambda z . y z x$
 2. Parenthèses autour des abstractions : $\lambda x . (\lambda y . (x y \lambda z . (y z x)))$
 3. Associativité applications : $\lambda x . (\lambda y . ((x y) (\lambda z . ((y z) x))))$
 4. Parenthèses externes : $(\lambda x . (\lambda y . ((x y) (\lambda z . ((y z) x))))$

4/Longueur d'un λ -terme :

notation $\text{lgh}(M)$

- **Variable** : si $x \in \mathcal{V} \rightarrow \text{lgh}(x) = 1$
- **Application** : si $M, N \in \Lambda \rightarrow \text{lgh}(M N) = \text{lgh}(M) + \text{lgh}(N)$
- **Abstraction** : si $x \in \mathcal{V}$ et $M \in \Lambda \rightarrow \text{lgh}(\lambda x . M) = \text{lgh}(M) + 1$

Exemple :

Terme : $M \equiv x \lambda y . y \cup x$

- Calcul : $\text{lgh}(M) = 5$

5/ Variables libres et liées

- **Notions :**

- Une **variable libre** n'est pas associée à une λ -abstraction.
- Une **variable liée** apparaît dans le corps d'une λ -abstraction correspondant à cette variable.
- Une variable peut être **libre et liée à la fois**.

- **Exemple :**

$M \equiv \lambda x . x \lambda y . y z x$

- Variables libres : $FV(M) = \{y, z\}$
- Variables liées : $BV(M) = \{x, y\}$

- **Définition inductive :**

- **Variables libres (FV) :**
 - $FV(x) = \{x\}$
 - $FV(M N) = FV(M) \cup FV(N)$
 - $FV(\lambda x . M) = FV(M) \setminus \{x\}$
- **Variables liées (BV) :**
 - $BV(x) = \emptyset$
 - $BV(M N) = BV(M) \cup BV(N)$
 - $BV(\lambda x . M) = BV(M) \cup \{x\}$

- **Remarques :**

- Si $x \notin (FV(M) \cup BV(M)) \rightarrow x$ n'apparaît pas dans M .
- Si $FV(M) = \emptyset \rightarrow M$ est **fermé** ou appelé **combinateur**.

6/ α -équivalence

- **Notion générale :**

- Deux fonctions sont équivalentes si seules les **noms des variables changent**.

- Mathématiques : $x \rightarrow f(x)$ et $y \rightarrow f(y)$
- λ -calcul : $\lambda x . x =_{\alpha} \lambda y . y$

- **Définition formelle :**

- α -équivalence est la **plus petite relation de congruence** sur les λ -termes telle que, pour tout M et toute variable y n'apparaissant pas dans M :
 - $\lambda x . M =_{\alpha} \lambda y . M[x:=y]$
- Ici, $M[x:=y]$ désigne la **substitution** de x par y dans M (définie plus bas).

Substitution : $M[x:=N]$

- Remplace **uniquement les occurrences libres** de x dans M par le λ -terme N .

- **Exemple :**

1. $(x (\lambda x y . x y z))[x:=N] \equiv N (\lambda x y . x y z)$
2. Attention à ne pas **capturer des variables libres** :
 - $M \equiv \lambda x . x y, N \equiv \lambda z . z x$

- $M[y:=N] \neq \lambda x . x (\lambda z . z x)$ directement
- Solution : renommer les variables liées : $M = \alpha \lambda x' . x' y \Rightarrow M[y:=N] \equiv \lambda x' . x' (\lambda z . z x)$

- **Définition inductive :**

1. $x[x:=N] \equiv N$
2. $y[x:=N] \equiv y$ si $x \neq y$
3. $(M P)[x:=N] \equiv (M[x:=N]) (P[x:=N])$
4. $(\lambda y . M)[x:=N] \equiv \lambda y . (M[x:=N])$ si $x \neq y$ et $y \notin FV(N)$
5. $(\lambda y . M)[x:=N] \equiv \lambda y' . (M[y:=y'] [x:=N])$ si $x \neq y$, $y \in FV(N)$, $y' \notin M$, $y' \notin N$

Propriétés de $=\alpha$:

- **Réflexivité :** $M =\alpha M$
- **Symétrie :** $M =\alpha N \Rightarrow N =\alpha M$
- **Transitivité :** $M =\alpha N$ et $N =\alpha P \Rightarrow M =\alpha P$
- **Congruence (passe au contexte) :**
 - $M =\alpha M'$ et $N =\alpha N' \Rightarrow M N =\alpha M' N'$
 - $M =\alpha N \Rightarrow \lambda x . M =\alpha \lambda x . N$

7/ β -réduction

- **Introduction :**
 - La β -réduction correspond à « évaluer » ou « exécuter » un λ -terme, un peu comme appliquer des arguments à une fonction en mathématiques.
 - Elle consiste à remplacer chaque **β -redex** par son équivalent.
- **Définition d'un β -redex :**
 - Un λ -terme de la forme $(\lambda x . M) N$ est un **β -redex**.
 - Un λ -terme sans β -redex est en **forme β -normale** ou **forme normale**.

Étape élémentaire : β -contraction

- Notation : $(\lambda x . M) N \rightarrow_{\beta} M[x:=N]$
- **Congruence :**
 - Si $M \rightarrow_{\beta} M'$, alors $MN \rightarrow_{\beta} M'N$
 - Si $N \rightarrow_{\beta} N'$, alors $MN \rightarrow_{\beta} MN'$
 - Si $M \rightarrow_{\beta} N$, alors $\lambda x . M \rightarrow_{\beta} \lambda x . N$

β -réduction complète :

- La β -réduction $\rightarrow_{\beta}$ est obtenue en appliquant **zéro ou plusieurs étapes élémentaires**.

Exemple :

- Terme : $(\lambda a . b)((\lambda c . ccc)(\lambda d . d))$

Méthode 1 :

1. $\rightarrow_{\beta} (\lambda a . b)((\lambda d . d)(\lambda d . d)(\lambda d . d))$ **$M = \lambda c . c c c$, $N = \lambda d . d$**
2. $\rightarrow_{\beta} (\lambda a . b)((\lambda d . d)(\lambda d . d)(\lambda d . d))$

3. $\rightarrow_{\beta} (\lambda a.b)((\lambda d.d)(\lambda d.d))$
4. $\rightarrow_{\beta} (\lambda a.b)(\lambda d.d)$ "Substituer 'a' par $\lambda d.d$ dans b. Comme b ne dépend pas de 'a', b reste inchangé"
5. $\rightarrow_{\beta} b$

Méthode 2 :

- Directement : $(\lambda a.b)((\lambda c.ccc)(\lambda d.d)) \rightarrow_{\beta} b$

Remarques :

- Le **nombre d'étapes** pour atteindre la forme normale dépend du **choix des redex** à réduire en premier.
- Réduire un redex peut **créer de nouveaux redex** ou en **supprimer certains**.

8/ β -abstraction

- **Définition :**
 - La β -abstraction est l'**inverse de la β -réduction**.
 - Elle consiste à **créer de nouveaux β -redex** à partir d'un λ -terme, sans réduire les redex existants.
 - Notation : $\beta\leftarrow$
- **Exemple :** $xxb\beta\leftarrow(\lambda a.aab)x$

9/ β -équivalence

- **Définition :**
 - La β -équivalence ($=_{\beta}$) relie deux λ -termes si on peut passer de l'un à l'autre par **β -réduction et/ou β -abstraction** (en plusieurs étapes ou aucune).
 - Exemple : $(\lambda a.b)((\lambda c.ccc)(\lambda d.d)) =_{\beta} b$
- **Propriétés :**
 - **Réflexivité :** $M =_{\beta} M$
 - **Symétrie :** $M =_{\beta} N \Rightarrow N =_{\beta} M$
 - **Transitivité :** $M =_{\beta} N$ et $N =_{\beta} P \Rightarrow M =_{\beta} P$
- **Congruence (passe au contexte) :**
 - Si $M =_{\beta} M'$ alors $M N =_{\beta} M' N$
 - Si $N =_{\beta} N'$ alors $M N =_{\beta} M N'$
 - Si $M =_{\beta} N$ alors $\lambda x.M =_{\beta} \lambda x.N$

10/ Stratégie de réduction

- **Définition :**
 - Une stratégie de réduction définit **quel redex réduire en premier**.
- **Types de stratégies :**
 - **Stratégie normale :**
 - Réduit les redex **les plus externes et les plus à gauche** en premier.
 - **Stratégie applicative :**
 - Réduit les redex **les plus internes et les plus à gauche** en premier.
- **Notions :**

- Redex externe : $(\lambda x.M)N$ est externe aux redex dans M et N.
- Redex internes : ceux à l'intérieur de M ou N.
- Plus externe : non contenu dans un autre redex.
- Plus interne : contenu dans aucun autre redex.

Exemple :

- Terme : $P \equiv (\lambda a. \lambda b. ((\lambda c. \lambda d. dc)a)((\lambda e. e)b))xy$
- **Stratégie normale :**
 - $\rightarrow \beta (\lambda b. ((\lambda c. \lambda d. d c) x) ((\lambda e. e) b)) y$
 - $\rightarrow \beta ((\lambda c. \lambda d. d c) x) ((\lambda e. e) y)$
 - $\rightarrow \beta (\lambda d. d x) ((\lambda e. e) y)$
 - $\rightarrow \beta ((\lambda e. e) y) x$
 - $\rightarrow \beta y x$
- **Stratégie applicative :**
 - $\rightarrow \beta (\lambda a. \lambda b. (\lambda d. d a) ((\lambda e. e) b)) x y$
 - $\rightarrow \beta (\lambda a. \lambda b. (\lambda d. d a) b) x y$
 - $\rightarrow \beta (\lambda a. \lambda b. b a) x y$
 - $\rightarrow \beta (\lambda b. b x) y$
 - $\rightarrow \beta y x$

11/ η -réduction

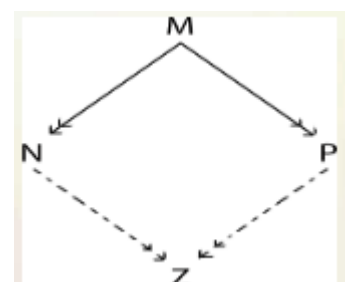
- **Définition :**
 - L'étape élémentaire : $\lambda x.M \rightarrow_{\eta} M$ si $x \notin FV(M)$
- On peut supprimer une abstraction inutile si la variable n'est pas utilisée dans M.
- **η -réduction complète :**
 - Notée $\rightarrow_{\eta}$: application de zéro ou plusieurs étapes.
 - **η -équivalence** : $=_{\eta}$
- **Forme normale :**
 - Un λ -terme est en **forme η -normale** s'il ne peut plus être réduit par η .

Relations combinées :

- $\rightarrow_{\beta\eta}$: une étape de β ou η réduction
- $\rightarrow_{\beta\eta}$: plusieurs étapes de β et/ou η
- $=_{\beta\eta}$: équivalence avec β -réduction, β -abstraction et η -réduction
- **Forme $\beta\eta$ -normale :**
 - Un terme est en forme $\beta\eta$ -normale s'il est à la fois :
 - en **forme β -normale**
 - et en **forme η -normale**

12/ Théorème de Church-Rosser

- **Théorème :**
 - Si un λ -terme M peut être réduit en deux termes N et P :
 $M \rightarrow N$ et $M \rightarrow P$
- Alors il existe un terme Z tel que : $N \rightarrow Z$ et $P \rightarrow Z$



Corollaires :

1. **Si $M =_{\beta} N$**
 - Il existe Z tel que : $M \rightarrow_{\beta} Z$ et $N \rightarrow_{\beta} Z$
2. **Forme normale unique :**
 - Si N est en forme β -normale et $N =_{\beta} M$ alors : $M \rightarrow_{\beta} N$
3. **Unicité (à α près) :**
 - Si M et N sont en forme β -normale et $M =_{\beta} N$ alors : $M =_{\alpha} N$
4. **Existence de forme normale :**
 - Si $M =_{\beta} N$ alors :
 - soit les deux ont une forme normale
 - soit aucun des deux n'en a
5. **Exemple :**
 - x et $\lambda y.xy$ ne sont pas β -équivalents
 - Car ils sont en forme normale mais **pas α -équivalents**

13/ À propos de la forme normale

- **Unicité :**
 - Si une forme normale existe, elle est **unique (à α près)** (théorème de Church-Rosser).
- **Existence :**
 - Une forme normale **n'existe pas toujours**.
 - Exemple : $M \equiv (\lambda x.xxx)(\lambda x.xxx)$
- Sa réduction ne s'arrête jamais $\rightarrow$ **pas de forme normale**.

Influence de la stratégie :

- Exemple : $N \equiv (\lambda y.z)((\lambda x.xxx)(\lambda x.xxx))$
- Si on réduit d'abord $M \rightarrow$ boucle infinie
- Si on réduit $(\lambda y.z) \rightarrow N \rightarrow_{\beta} z$
- Donc une forme normale peut exister mais **dépendre de la stratégie**.

Types de normalisation :

- **Fortement normalisable :**
 - Toutes les réductions sont **finies** (quel que soit l'ordre).
- **Faiblement normalisable :**
 - Il existe **au moins une réduction finie**.

Exemples :

- $(\lambda x.xy)(\lambda z.zw) \rightarrow$ **fortement normalisable**
- $(\lambda y.z)((\lambda x.xxx)(\lambda x.xxx)) \rightarrow$ **faiblement normalisable**
- $(\lambda x.xx)(\lambda x.xx) \rightarrow$ **non normalisable**

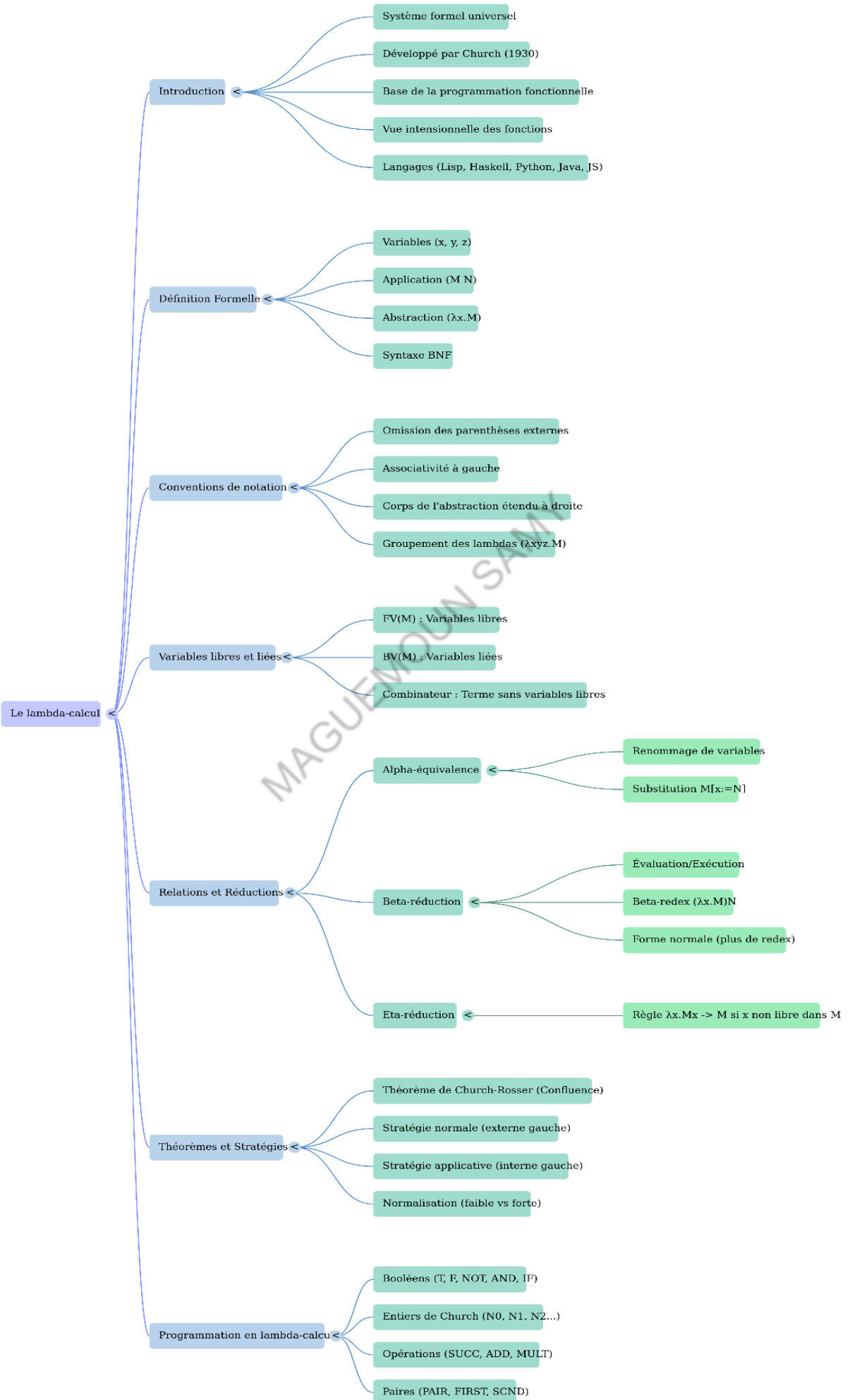
Remarques :

- **Fortement normalisable :** toutes les réductions **terminent toujours** vers une forme normale.

- **Faiblement normalisable** : certains chemins **terminent**, d'autres peuvent **ne jamais terminer**.
- La forme normale, si elle existe, est **unique** (pas plusieurs résultats différents)

14/ Programmation en λ -calcul

- **Booléens** :
 - Vrai : $T \equiv \lambda x y . x$
 - Faux : $F \equiv \lambda x y . y$
 - Non : $NOT \equiv \lambda p . p F T$
 - Et : $AND \equiv \lambda p q . p q F$
 - Condition (if) : $IF \equiv \lambda p q . p q (NOT q)$
- **Entiers (entiers de Church)** :
 - $N0 \equiv \lambda f x . x$
 - $N1 \equiv \lambda f x . f x$
 - $N2 \equiv \lambda f x . f (f x)$
 - $N3 \equiv \lambda f x . f (f (f x))$
- **Opérations** :
 - Successeur : $SUCC \equiv \lambda n f x . f (n f x)$
 - Addition : $ADD \equiv \lambda n x . n SUCC x$
 - Multiplication : $MULT \equiv \lambda n x f . n (x f)$
- **Paires** :
 - Définition : $PAIR \equiv \lambda z . z M N$
 - Premier élément : $FIRST \equiv \lambda p . p (\lambda x y . x)$
 - Deuxième élément : $SCND \equiv \lambda p . p (\lambda x y . y)$



Chapitre 2 : Théorie des combinateurs

1/ Introduction à la théorie des combinateurs

- Système formel (Schönfinkel, Curry) **sans variables** → pas de problèmes de substitution.
- Basé sur des **constantes atomiques** (A, B, C...).
- Règles :
 - Une constante est un combinateur.
 - Si A et B sont des combinateurs → (A B) est un combinateur.
- Application **associative à gauche** : $(A B C) = ((A B) C)$

2/ Correspondance avec le λ -calcul

- **Définition** :
 - Un combinateur = **λ -terme sans variables libres**.
- **Exemples** :
 - Combinateurs : $\lambda x.x$, $\lambda xy.y$, $\lambda x.x(\lambda y.yx)$
 - Non combinateurs : $y \lambda x.x$, $\lambda xy.z$, $\lambda x.xy x$
- **Question** :
 - On peut représenter le λ -calcul avec un **nombre fini de constantes atomiques** (ex : S, K, I).

3/ Éléments de réponse

- Pour relier combinateurs et λ -calcul, il faut remplacer la **λ -abstraction** par des **constantes atomiques**.
- L'objectif : traduire tout λ -terme **sans variables libres** en combinateurs (et inversement).
- Il faut donc pouvoir représenter des formes comme :
 - $\lambda x . M$
 - $\lambda x . M x$
 - $\lambda x . \lambda y . N M$
- Avec :
 - M : combinateur
 - N : λ -terme avec $FV(N) = \{x\}$
 - x : variable

4/ Les combinateurs SKI

- **Définition** :
 - $S \equiv \lambda f g x . f x (g x)$
 - $K \equiv \lambda x y . x$
 - $I \equiv \lambda x . x$
- **Rôle** :
 - S : applique une fonction à deux résultats
 - K : retourne une constante
 - I : identité
- **Proposition** :

- Les combinateurs **S, K, I** suffisent pour représenter tout λ -terme fermé.

5/ Combinateurs SKI : correspondance

- **But :**
 - Transformer un λ -terme en une expression avec **S, K, I uniquement** (fonction T).

Règles de transformation T :

1. $T[x] \equiv x$ (si x est une variable ou une constante atomique S, K, I)
2. $T[\lambda x . x] \equiv I, T[\lambda x y . x] \equiv K$ et $T[\lambda f g x . f x (g x)] \equiv S$
3. $T[M N] \equiv T[M] T[N]$
4. $T[\lambda x . M] \equiv K T[M]$ si $x \notin FV(M)$
5. $T[\lambda x y . M] \equiv T[\lambda x . T[\lambda y . M]]$ si $x \in FV(M)$
6. $T[\lambda x . (M N)] \equiv S T[\lambda x . M] T[\lambda x . N]$ si $x \in FV(M)$ ou $x \in FV(N)$

- **Exemples :**

- $\lambda x . x(\lambda y . y) \rightarrow S I (K I)$
- $\lambda x y . y x \rightarrow S (K (S I)) (S (K K) I)$

- **Sens inverse :**

- On peut aussi revenir au λ -calcul par **β -réduction**.
- Exemple : $S I (K I) = \lambda x . x (\lambda y . y)$

6/ Exemples de correspondance SKI

λ -calcul $\rightarrow$ SKI

Exemple 1 : $T[\lambda x . x(\lambda y . y)]$

1. $\equiv S T[\lambda x . x] T[\lambda x . (\lambda y . y)]$ (règle 6)
2. $\equiv S I (K T[\lambda y . y])$ (règle 2 et règle 4)
3. $\equiv S I (K I)$ (règle 2)

Exemple 2 : $T[\lambda x y . yx]$

1. $\equiv T[\lambda x . T[\lambda y . y x]]$ (règle 5)
2. $\equiv T[\lambda x . S T[\lambda y . y] T[\lambda y . x]]$ (règle 6)
3. $\equiv T[\lambda x . S I T[\lambda y . x]]$ (règle 2)
4. $\equiv T[\lambda x . S I (K x)]$ (règle 4)
5. $\equiv S T[\lambda x . (S I)] T[\lambda x . K x]$ (règle 6)
6. $\equiv S (K (S I)) T[\lambda x . K x]$ (règle 4)
7. $\equiv S (K (S I)) (S T[\lambda x . K] T[\lambda x . x])$ (règle 6)
8. $\equiv S (K (S I)) (S (K K) I)$ (règle 4 et règle 2)

SKI $\rightarrow$ λ -calcul

Exemple : $SI(KI)$

1. $\equiv (\lambda f g x . f x (g x)) (\lambda x . x) ((\lambda x y . x)(\lambda x . x))$ (définition de S, K, I)
2. $\rightarrow \beta (\lambda f g x . f x (g x)) (\lambda x . x) (\lambda y x . x)$ (β -réduction)

3. $\rightarrow\beta (\lambda g x . (\lambda x.x) x (g x)) (\lambda y x . x)$ (β -réduction)
4. $\rightarrow\beta (\lambda x . (\lambda x.x) x ((\lambda y x . x) x))$ (β -réduction)
5. $\rightarrow\beta (\lambda x . x ((\lambda y x . x) x))$ (β -réduction)
6. $\rightarrow\beta (\lambda x . x (\lambda x.x))$ (β -réduction)
7. $=\alpha (\lambda x . x (\lambda y.y))$ (α -équivalence)

7/Combinateurs SKI : Définition et Réduction

- **Ensemble de base** : CSKI construit à partir des constantes atomiques **S**, **K**, **I**.
- **Règles de réduction élémentaire** (pour tout $M, N, P \in \text{CSKI}$)
 - $SMNP \rightarrow MP(NP)$
 - $KMN \rightarrow M$
 - $IM \rightarrow M$
- **Symboles** :
 - $\rightarrow$: une étape élémentaire de réduction
 - $\rightarrow^*$: zéro ou plusieurs étapes de réduction
- **Compatibilité avec le λ -calcul** :
 - $SMNP \equiv (\lambda f g x . f x (g x)) MNP \rightarrow \beta MP(NP)$
 - $KMN \equiv (\lambda x y . x) MN \rightarrow \beta M$
 - $IM \equiv (\lambda x . x) M \rightarrow \beta M$
- **Remarque importante** :
Comme M, N, PM, N, PM, N, P sont des combinateurs, ils n'ont **pas de variables libres**, donc toutes les substitutions sont triviales :
 $M[g:=N] \equiv M, M[x:=P] \equiv M, N[x:=P] \equiv N, M[y:=N] \equiv M$

Exemple : Réduire dans le système des combinateurs SKI le combinateur suivant : $S (S K I I) K (S I) I$

$S (S K I I) K (S I) I$	$\rightarrow S$	(application S)
$(S K I I) (S I) (K (S I)) I$	$\rightarrow S$	(application S)
$(K I (I I)) (S I) (K (S I)) I$	$\rightarrow K$	(application K)
$(K I I) (S I) (K (S I)) I$	$\rightarrow K$	(application K)
$I (S I) (K (S I)) I$	$\rightarrow I$	(application I)
$(S I) (K (S I)) I$	$\rightarrow S$	(associativité gauche)
$S I (K (S I)) I$	$\rightarrow S$	(application S)
$I I ((K (S I)) I)$	$\rightarrow I$	(application I)
$I ((K (S I)) I)$	$\rightarrow I$	(application I)
$(K (S I)) I$	$\rightarrow K$	(application K)
$S I$		fin

8/Combinateurs BCKW :

Définition des combinateurs BCKW

- **B (composition)** : $B \equiv \lambda f g x . f(gx)$
- **C (inversion des arguments)** : $C \equiv \lambda f x y . fyx$
- **K (constante)** : $K \equiv \lambda x y . x$
- **W (récursion / duplication)** : $W \equiv \lambda x y . xyy$

Remarques :

- I (identité) peut être construit comme : $I \equiv WK$
- S et K peuvent être exprimés avec B, C, W :
 - $S \equiv B(B(BW)C)(BB) \equiv B(BW)(BBC)$
 - $K \equiv K$

Alternative : représentation BCKW en combinateurs SKI :

- $B \equiv S(KS)K$
- $C \equiv S(S(K(S(KS)K))S)(KK)$
- $K \equiv K$
- $W \equiv SS(SK)$

Conversion λ -calcul $\rightarrow$ BCKW

1. D'abord, convertir le λ -terme en combinateurs SKI.
2. Remplacer ensuite chaque constante SKI par son équivalent BCKW :

SKI	BCKW équivalent
S	$B(BW)(BBC)$
K	K
I	WK

Exemple : λ -terme : $\lambda x.x(\lambda y.y)$

1. Conversion SKI : $T[\lambda x.x(\lambda y.y)] \equiv SI(KI)$
2. Conversion BCKW : $SI(KI) = \beta(B(BW)(BBC))(WK)(K(WK))$

9/Combinateur de point fixe :

- **Maths** : x est un point fixe de f si $f(x)=x$.
- **λ -calcul / combinateurs** : N est un point fixe de F si $FN=\beta N$.

Théorème :

- **Énoncé** : Tout λ -terme F a toujours un point fixe N tel que $FN=\beta$.
- **Preuve / idée** : On peut construire un λ -terme capable de calculer le point fixe pour n'importe quel F.
- **Historique** : Résolu par Curry et Rosser, puis par Alan Turing.
- **Curry-Rosser** :
 - $Y \equiv \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$
 - Ici, $V \equiv \lambda x.f(xx)$ et $Y \equiv \lambda f.VV$
- **Turing** :
 - $\Theta \equiv (\lambda xy.y(xxy))(\lambda xy.y(xxy))$
 - Ici, $U \equiv \lambda xy.y(xxy)$ et $\Theta \equiv UU$

- **Propriété générale :**

Si Z est un combinateur de point fixe, alors (ZF) est un point fixe de F , c'est-à-dire:
 $(ZF) = \beta F(ZF)$

Preuve des combinateurs de point fixe – Résumé :

- **Curry-Rosser :**

- $Y F \equiv (\lambda f. (\lambda x. f(xx)) (\lambda x. f(xx))) F$
- $\rightarrow \beta (\lambda x. F(xx)) (\lambda x. F(xx))$
- $\rightarrow \beta F ((\lambda x. F(xx)) (\lambda x. F(xx)))$
- $\beta \leftarrow F ((\lambda f. (\lambda x. f(xx)) (\lambda x. f(xx))) F)$
- $\equiv F (Y F)$

Donc (YF) est un point fixe de F .

- **Turing :**

- $\Theta F \equiv (\lambda xy. y(xxy)) (\lambda xy. y(xxy)) F$
- $\rightarrow \beta (\lambda y. y((\lambda xy. y(xxy)) (\lambda xy. y(xxy)) y)) F$
- $\rightarrow \beta F ((\lambda xy. y(xxy)) (\lambda xy. y(xxy)) F)$
- $\equiv F (\Theta F)$

Donc (ΘF) est un point fixe de F .

Exemple de point fixe :

- **Terme à fixer :** $F \equiv \lambda ab. a$

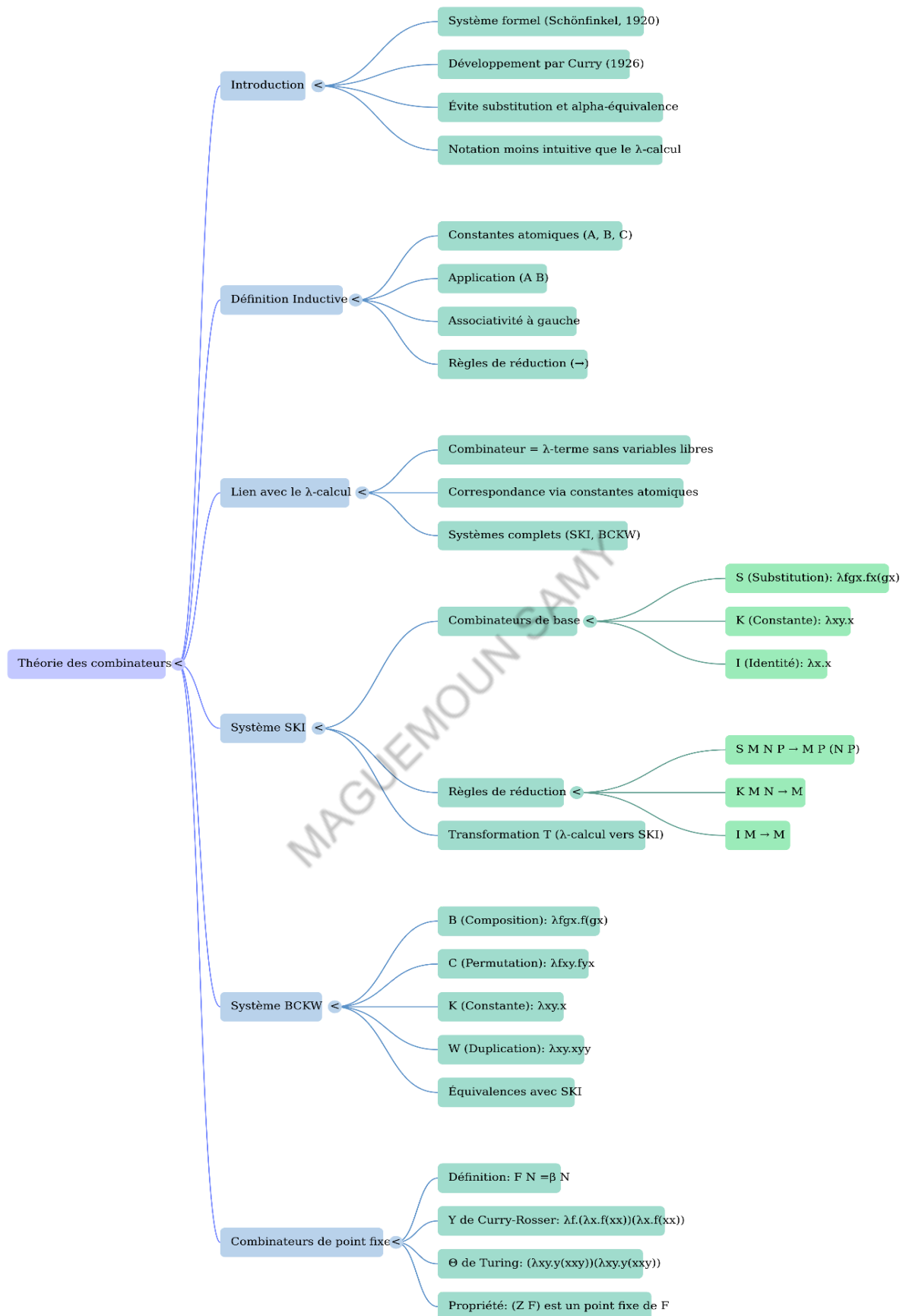
- **Combinateur de point fixe de Curry :**

- $Y F \equiv (\lambda f. (\lambda x. f(xx)) (\lambda x. f(xx))) (\lambda ab. a)$
- $\rightarrow \beta (\lambda x. (\lambda ab. a)(xx)) (\lambda x. (\lambda ab. a)(xx))$
- $\rightarrow \beta (\lambda xb. xx)(\lambda xb. xx) \equiv N$

- **Vérification :**

- $F N \equiv (\lambda ab. a) ((\lambda xb. xx)(\lambda xb. xx))$
- $\rightarrow \beta (\lambda b. (\lambda xb. xx)(\lambda xb. xx))$
- $\beta \leftarrow (\lambda xb. xx)(\lambda xb. xx) \equiv N$

Donc $N = (\lambda xb. xx)(\lambda xb. xx)$ est bien un point fixe de F .



Chapitre 3 : Théorie des types

1/ Introduction (λ -calcul typé)

- En mathématiques, une fonction a un **domaine** (entrée) et un **codomaine** (sortie).
- De même, en λ -calcul, on peut associer des **types** aux λ -termes pour préciser entrées/sorties.
- Deux approches de typage :
 - **Approche explicite (Church)**
 - **Approche implicite (Curry)**
- **Intérêt du typage** :
 - Garantit que les λ -termes bien typés sont **fortement normalisables** (les calculs terminent toujours).
 - Empêche des termes problématiques comme : $(\lambda x.xx)(\lambda x.xx)$
- Permet d'éviter certains paradoxes logiques, notamment le **paradoxe de Russell**.

2/ Paradoxe de Russell (en λ -calcul)

- Un ensemble peut être représenté par une **fonction caractéristique** :
 - $FE(x)=\text{vrai}$ si $x \in E$, sinon faux
- L'ensemble problématique $R=\{x \mid x \notin x\}$ devient en λ -calcul $R \equiv \lambda x. \text{NOT}(xx)$
- En testant R sur lui-même : $RR \rightarrow \beta \text{NOT}(RR)$
- **Contradiction** : $RR = \beta \text{NOT}(RR)$
→ impossible d'être vrai et faux en même temps
- **Conclusion** :
Ce paradoxe montre l'importance du **typage**, qui empêche ce type de construction incohérente.

3/ Types

Définition :

- Soit un ensemble $\mathcal{T}_{\text{atom}}$ de **types atomiques**.
- L'ensemble des types $\mathcal{T}_{\text{const}}$ est défini par :
 1. Tout type atomique est un type.
 2. Si σ et τ sont des types, alors $(\sigma \rightarrow \tau)$ (type fonction) est un type.

Conventions :

- Lettres majuscules (A, B, ...) : types atomiques.
- Lettres grecques (σ , τ , ...) : types quelconques.
- Associativité à droite : $\sigma \rightarrow \tau \rightarrow \upsilon \equiv \sigma \rightarrow (\tau \rightarrow \upsilon)$

4/ Types paramétriques

Définition :

- Soient :
 1. $\mathcal{T}var$: ensemble de **variables de types**
 2. $\mathcal{T}atom$: ensemble de **types atomiques**
- L'ensemble des types paramétriques $\mathcal{T}param$ est défini par :
 1. Tout élément de $\mathcal{T}var$ ou $\mathcal{T}atom$ est un type.
 2. Si σ et τ sont des types, alors $(\sigma \rightarrow \tau)$ est un type paramétrique.

Conventions :

- Lettres minuscules (a, b, ...) : variables de types.
- Lettres majuscules (A, B, ...) : types atomiques.
- Lettres grecques (σ , τ , ...) : types quelconques.
- Associativité à droite : $\sigma \rightarrow \tau \rightarrow u \equiv \sigma \rightarrow (\tau \rightarrow u)$

Remarques :

- **Type fermé** : contient uniquement des types atomiques.
- **Type ouvert** : contient des variables de types (ou un mélange).

Exemples

- Soient :
 - N : type des entiers
 - H : type des booléens
- $N \rightarrow N$: fonction prenant un entier et retournant un entier.
- $N \rightarrow H$: fonction prenant un entier et retournant un booléen.
- $(N \rightarrow N) \rightarrow H$: fonction prenant une fonction en entrée et retournant un booléen.
- $(N \rightarrow N) \rightarrow (H \rightarrow H)$: fonction prenant une fonction et retournant une autre fonction.

Avec types paramétriques :

- $a \rightarrow a$: fonction qui prend une valeur et retourne une valeur du **même type** (ex : $N \rightarrow N$, $H \rightarrow H$, etc.).

5/ Variables typées

Définition :

- On part d'un ensemble de variables non typées $\mathcal{V}(x, y, z, \dots)$.
- On leur associe des types $\sigma, \tau, \dots$ pour obtenir des **variables typées** : x^σ, y^τ, z^u .

Règles :

1. **Cohérence** : une variable a un seul type
→ on ne peut pas avoir x^σ et x^τ avec $\sigma \neq \tau$.

2. **Partage** : plusieurs variables peuvent avoir le même type.

6/ λ -calcul simplement typé (Church)

Définition :

- On associe un **type à chaque variable directement** (typage explicite).
- Les λ -termes typés Λ Church sont définis par :
 1. **Variable** :
 - Si $x \in \mathcal{V}$ et $\sigma \in \mathcal{T}$, alors x^σ est un terme de type σ .
 2. **Application** :
 - Si $M^\sigma \rightarrow \tau$ et N^σ , alors : $(M N)^\tau$
 3. **Abstraction** :
 - Si x^σ et M^τ , alors : $(\lambda x^\sigma. M)^\sigma \rightarrow \tau$

Exemple :

- Terme bien typé : $\lambda x^\sigma \{ \sigma \rightarrow \sigma \} . x ((\lambda y^\tau . z^\tau \{ \tau \rightarrow \sigma \} y) w^\tau \{ \tau \})$

Remarque :

- On peut **omettre les types** lorsqu'ils sont évidents (inférables).
- Ici, les types servent à garantir la **cohérence et la terminaison** des calculs.

7/ λ -calcul simplement typé (Curry)

Définition :

- Contrairement à Church, le typage est **implicite** (les types ne sont pas écrits dans les termes).
- On utilise un **environnement de types** Γ qui associe des types aux variables.
- Les λ -termes sont non typés, mais on leur attribue un type via Γ .

Notion clé :

- $\Gamma \vdash M : \sigma$ signifie :
 - dans le contexte Γ , le terme M a le type σ .

Remarques :

- Γ, Δ : environnements de types.
- Cohérence : pas de conflit de types pour une même variable.
- Les types des variables liées peuvent être **inférés** (pas besoin de les écrire).

Exemple :

- $\Gamma = \{w^\tau, z^\tau \{ \tau \rightarrow \sigma \} \}$
- Alors : $\Gamma \vdash \lambda x. x((\lambda y. zy) w) : (\sigma \rightarrow u) \rightarrow u$

8/ Comparaison Church vs Curry

λ -calcul typé à la Church :

- Typage **explicite** (types écrits dans les termes).
- Chaque variable a un type précis.
- Une même fonction peut avoir **plusieurs écritures** selon le type : $\lambda x^\sigma. x$, $\lambda x^\{\sigma \rightarrow \sigma\}. x, \dots$

λ -calcul typé à la Curry :

- Typage **implicite** (types déduits automatiquement).
- Les types ne sont pas écrits dans les termes.
- Une fonction a **une seule écriture générale** : $\vdash \lambda x. x : \sigma \rightarrow \sigma$

9/ λ -calcul simplement typé : Propriétés

1. Substitution :

- $M^\tau[x^\sigma := N^\sigma]$: substitution valide $\rightarrow$ le type τ est **préservé**.
- $M^\tau[x^\sigma := N^\sigma]$: **non définie** si $\sigma \neq \tau$.

2. α -équivalence :

- Identique au λ -calcul non typé.
- Respecte les types lors des substitutions.

3. β -réduction :

- Même principe que non typé, avec respect des types : $(\lambda x^\sigma. M^\tau)N^\sigma \rightarrow_\beta M^\tau[x^\sigma := N^\sigma]$
- La β -équivalence est définie de la même manière.

4. η -réduction :

- Ne **préserve pas les types** : $(\lambda x^\sigma. M^\tau)^\{\sigma \rightarrow \tau\} \rightarrow_\eta M^\tau$
- Le type change ($\sigma \rightarrow \tau \neq \tau$).

10/ Règles de typage

1. Variable (var) : $\frac{}{\Gamma, x^\sigma \vdash x : \sigma}$

$\rightarrow$ Une variable a le type donné dans le contexte.

2. Abstraction (abs) : $\frac{\Gamma, x^\sigma \vdash M : \tau}{\Gamma \vdash \lambda x. M : \sigma \rightarrow \tau}$

$\rightarrow$ Une fonction prend un type σ et retourne τ .

3. Application (app) : $\frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Delta \vdash N : \sigma}{\Gamma \cup \Delta \vdash MN : \tau}$

$\rightarrow$ On applique une fonction à un argument du bon type.

Remarques :

- $\Gamma, x^\Delta \sigma \equiv \Gamma \cup \{x^\Delta \sigma\}$
- Si $\Gamma \cup \Delta$ n'est pas valide $\rightarrow$ terme non typé.
- Si $\Gamma \subseteq \Delta$ alors : $\Gamma \vdash M : \sigma \Rightarrow \Delta \vdash M : \sigma$

Exemple 1 : $\lambda xyz. x(yz)$

$$\begin{array}{c}
 \text{(var)} \frac{}{y^{v \rightarrow \tau} \vdash y : v \rightarrow \tau} \quad \text{(var)} \frac{}{z^v \vdash z : v} \\
 \text{(var)} \frac{}{x^{\tau \rightarrow \sigma} \vdash x : \tau \rightarrow \sigma} \quad \text{(app)} \frac{}{y^{v \rightarrow \tau}, z^v \vdash yz : \tau} \\
 \text{(app)} \frac{}{x^{\tau \rightarrow \sigma}, y^{v \rightarrow \tau}, z^v \vdash x(yz) : \sigma} \\
 \text{(abs)} \frac{}{x^{\tau \rightarrow \sigma}, y^{v \rightarrow \tau} \vdash \lambda z. x(yz) : v \rightarrow \sigma} \\
 \text{(abs)} \frac{}{x^{\tau \rightarrow \sigma} \vdash \lambda yz. x(yz) : (v \rightarrow \tau) \rightarrow v \rightarrow \sigma} \\
 \text{(abs)} \frac{}{\vdash \lambda xyz. x(yz) : (\tau \rightarrow \sigma) \rightarrow (v \rightarrow \tau) \rightarrow v \rightarrow \sigma}
 \end{array}$$

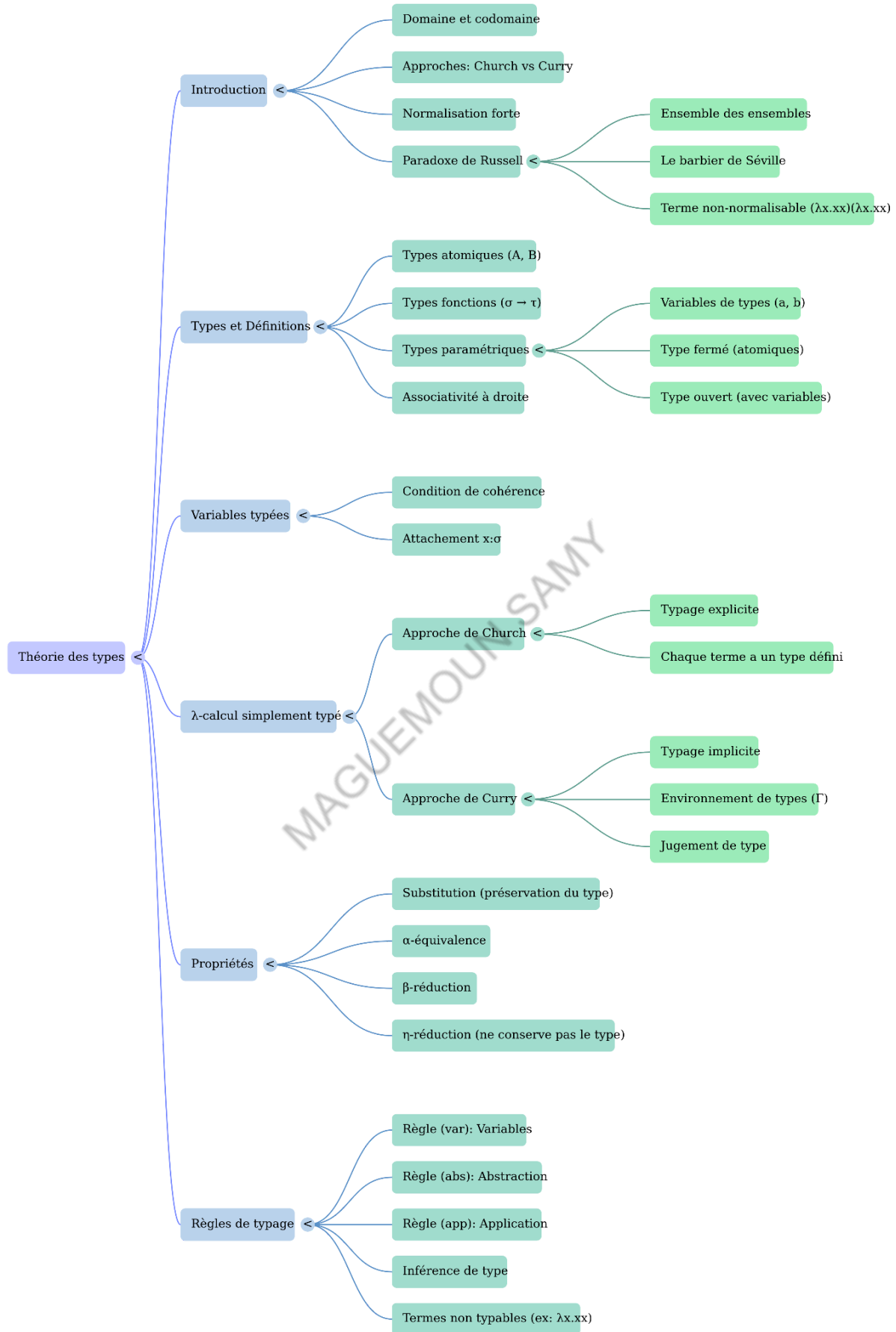
Exemple 2 : $\lambda x.xx$

$$\begin{array}{c}
 \text{(var)} \frac{}{x^{\tau \rightarrow \sigma} \vdash x : \tau \rightarrow \sigma} \quad \text{(var)} \frac{}{x^\tau \vdash x : \tau} \\
 \text{(app)} \frac{}{x^\tau, x^{\tau \rightarrow \sigma} \vdash xx : \sigma}
 \end{array}$$

Impossible de résoudre : x aurait deux types différents $\rightarrow$ non typable

Exemple 3 : $\vdash \lambda fx. f(fx) : (\sigma \rightarrow \sigma) \rightarrow \sigma \rightarrow \sigma$

$$\begin{array}{c}
 \vdash \lambda fx. f(fx) : (\sigma \rightarrow \sigma) \rightarrow \sigma \rightarrow \sigma \\
 \text{(abs)} \frac{}{f^{(\sigma \rightarrow \sigma)} \vdash \lambda x. f(fx) : \sigma \rightarrow \sigma} \\
 \text{(abs)} \frac{}{f^{(\sigma \rightarrow \sigma)}, x^\sigma \vdash f(fx) : \sigma} \\
 \text{(app)} \frac{}{f^{(\sigma \rightarrow \sigma)} \vdash f : \sigma \rightarrow \sigma \quad f^{(\sigma \rightarrow \sigma)}, x^\sigma \vdash fx : \sigma} \\
 \text{(app)} \frac{}{f^{(\sigma \rightarrow \sigma)} \vdash f : \sigma \rightarrow \sigma \quad x^\sigma \vdash x : \sigma}
 \end{array}$$



Chapitre 4 : Théorème de Curry-Howard

1/ Introduction (Curry-Howard)

- Le **théorème de Curry-Howard** établit une correspondance entre **λ -calcul typé** et **logique intuitionniste**.
- **Idée principale** :
 - Une **preuve** = un **programme**
 - Une **proposition** = un **type**
- Une proposition prouvable $\Leftrightarrow$ un programme qui s'exécute sans erreur.
- **Intérêt** :
 - Vérifier la **correction des programmes**
 - Lier informatique et mathématiques (un programme peut être vu comme une preuve)

2/ $\lambda\pi$ -calcul typé

Définition :

- Le $\lambda\pi$ -calcul typé étend le λ -calcul typé en ajoutant :
 - les **types produit** (paires)
 - les **types somme**

2.1/types produit et somme

Type produit (paire)

- Construction : (M, N) de type $\sigma \times \tau$
- Projections :
 - $\pi_1(M, N)^{\{\sigma \times \tau\}} \rightarrow_{\beta} M^{\sigma}$
 - $\pi_2(M, N)^{\{\sigma \times \tau\}} \rightarrow_{\beta} N^{\tau}$

Type somme (variant)

- Construction :
 - $\text{in}_1\tau(M) : \text{type } \sigma + \tau$
 - $\text{in}_2\tau(M) : \text{type } \tau + \sigma$
- Élimination (analyse de cas) :
 - $\text{case}(\text{in}_1^{\tau}(M); x.N; y.P) \rightarrow_{\beta} N[x := M]$
 - $\text{case}(\text{in}_2^{\tau}(M); x.N; y.P) \rightarrow_{\beta} P[y := M]$

2.2/règles de typage

Type produit (paire)

$$\text{(paire)} \frac{\Gamma \vdash M : \sigma \quad \Delta \vdash N : \tau}{\Gamma \cup \Delta \vdash (M, N) : \sigma \times \tau}$$

Type somme (variant)

$$(\pi_1) \frac{\Gamma \vdash M : \sigma \times \tau}{\Gamma \vdash \pi_1 M : \sigma}$$

$$(\pi_2) \frac{\Gamma \vdash M : \sigma \times \tau}{\Gamma \vdash \pi_2 M : \tau}$$

$$\begin{array}{c}
 \text{(in}_1\text{)} \frac{\Gamma \vdash M : \sigma}{\Gamma \vdash \text{in}_1^\tau(M) : \sigma + \tau} \\
 \\
 \text{(in}_2\text{)} \frac{\Gamma \vdash M : \sigma}{\Gamma \vdash \text{in}_2^\tau(M) : \tau + \sigma} \\
 \\
 \text{(case)} \frac{\Gamma \vdash M : \sigma + \tau \quad \Gamma, x^\sigma \vdash N : v \quad \Gamma, y^\tau \vdash P : v}{\Gamma \vdash \text{case}(M; x. N; y. P) : v}
 \end{array}$$

Idée rapide :

- **paire** $\rightarrow$ construit (M, N)
- π_1 / π_2 $\rightarrow$ extraire
- **in1** / **in2** $\rightarrow$ choisir un cas
- **case** $\rightarrow$ traiter les deux cas possibles

Remarques

- **Priorité des opérateurs :**
 - $\times$ et $+$ sont **plus prioritaires que** $\rightarrow$
 - Donc :

$$\sigma \times \tau \rightarrow u \equiv (\sigma \times \tau) \rightarrow u$$

$$\sigma \rightarrow \tau \times u \equiv \sigma \rightarrow (\tau \times u)$$

$$\sigma + \tau \rightarrow u \equiv (\sigma + \tau) \rightarrow u$$

$$\sigma \rightarrow \tau + u \equiv \sigma \rightarrow (\tau + u)$$
- **Entre $\times$ et $+$:**
 - Ils ont **la même priorité**
 - $\Rightarrow$ il faut **toujours utiliser des parenthèses**
Exemple : $\sigma \times (\tau + u) \neq (\sigma \times \tau) + u$

Exemple 1 : $\lambda xyz . x(y, z)$	Exemple 2 : $\lambda x . (\pi_2 x, \pi_1 x) : \sigma \times \tau \rightarrow \tau \times \sigma$
$ \begin{array}{c} \text{(var)} \frac{}{x^{\sigma \times \tau \rightarrow v} \vdash x : \sigma \times \tau \rightarrow v} \quad \text{(var)} \frac{}{y^{\sigma} \vdash y : \sigma} \quad \text{(var)} \frac{}{z^{\tau} \vdash z : \tau} \\ \text{(app)} \frac{}{x^{\sigma \times \tau \rightarrow v}, y^{\sigma}, z^{\tau} \vdash x(y, z) : v} \\ \text{(abs)} \frac{}{x^{\sigma \times \tau \rightarrow v}, y^{\sigma} \vdash \lambda z . x(y, z) : \tau \rightarrow v} \\ \text{(abs)} \frac{}{x^{\sigma \times \tau \rightarrow v} \vdash \lambda yz . x(y, z) : \sigma \rightarrow \tau \rightarrow v} \\ \text{(abs)} \frac{}{\vdash \lambda xyz . x(y, z) : (\sigma \times \tau \rightarrow v) \rightarrow \sigma \rightarrow \tau \rightarrow v} \end{array} $	$ \begin{array}{c} \vdash \lambda x . (\pi_2 x, \pi_1 x) : \sigma \times \tau \rightarrow \tau \times \sigma \\ \text{(abs)} \frac{}{x^{\sigma \times \tau} \vdash (\pi_2 x, \pi_1 x) : \tau \times \sigma} \\ \text{(paire)} \frac{}{x^{\sigma \times \tau} \vdash \pi_2 x : \tau \quad x^{\sigma \times \tau} \vdash \pi_1 x : \sigma} \\ \text{(\pi}_2\text{)} \frac{}{x^{\sigma \times \tau} \vdash \pi_2 x : \tau} \quad \text{(\pi}_1\text{)} \frac{}{x^{\sigma \times \tau} \vdash \pi_1 x : \sigma} \end{array} $

3/ Logique intuitionniste : rappels

Logique classique :

- Existence : prouver que la non-existence est absurde suffit
- Tiers exclu : $A \vee \neg A$ toujours vrai

Logique intuitionniste :

- Existence : il faut **construire explicitement** l'objet
- $A \vee \neg A$ non prouvable
- $\neg \neg A \Rightarrow A$ non prouvable

Interprétation des propositions

- **A**
Classique : A est vraie
Intuitionniste : A est prouvable
- **$\neg A$**
Classique : A est fausse
Intuitionniste : A est contradictoire
- **$A \vee B$**
Classique : A ou B est vraie
Intuitionniste : A est prouvable ou B est prouvable
- **$A \wedge B$**
Classique : A et B sont vraies
Intuitionniste : A et B sont prouvables
- **$A \Rightarrow B$**
Classique : si A est vraie alors B est vraie
Intuitionniste : si A est prouvable alors B est prouvable

Quantificateurs

- $\exists x A(x)$
Classique : il existe x tel que $A(x)$ est vraie
Intuitionniste : on peut **construire** x tel que $A(x)$ est prouvable
- $\forall x A(x)$
Classique : pour tout x , $A(x)$ est vraie
Intuitionniste : pour tout x , $A(x)$ est prouvable

Exemple

- **Proposition** : $\exists x, y$ irrationnels tels que x^y est rationnel
- **Preuve non-constructive** :
raisonnement par cas sans exemple précis
- **Preuve constructive** :
 $x = \sqrt{2}, y = 2 \log_2 3 \Rightarrow x^y = 3$

Morale :

→ le constructivisme donne des **solutions explicites**

Règles :

• **Axiome :**

$$\frac{}{\Gamma, \phi \vdash \phi} (Ax)$$

• **Introduction :**

$\frac{\Gamma \vdash \phi \quad \Delta \vdash \psi}{\Gamma \cup \Delta \vdash \phi \wedge \psi} (\wedge I)$	$\frac{\Gamma \vdash \phi}{\Gamma \vdash \phi \vee \psi} (\vee I_1)$	$\frac{\Gamma \vdash \phi}{\Gamma \vdash \psi \vee \phi} (\vee I_2)$	$\frac{\Gamma, \phi \vdash \psi}{\Gamma \vdash \phi \Rightarrow \psi} (\Rightarrow I)$
---	--	--	--

• **Élimination :**

$\frac{\Gamma \vdash \phi \wedge \psi}{\Gamma \vdash \phi} (\wedge E_1)$	$\frac{\Gamma \vdash \phi \wedge \psi}{\Gamma \vdash \psi} (\wedge E_2)$	$\frac{\Gamma \vdash \phi \Rightarrow \psi \quad \Gamma \vdash \phi}{\Gamma \vdash \psi} (\Rightarrow E)$
--	--	---

$\frac{\Gamma \vdash \phi \vee \psi \quad \Gamma, \phi \vdash \omega \quad \Gamma, \psi \vdash \omega}{\Gamma \vdash \omega} (\vee E)$
--

• Résumé des types de règles :

1. **Axiome (Ax)** : $\Gamma, \phi \vdash \phi$
2. **Introduction (I)** : pour $\wedge, \vee, \Rightarrow$
3. **Élimination (E)** : pour $\wedge, \vee, \Rightarrow$

4/Correspondance de Curry-Howard

Théorème principal :

Les preuves en logique intuitionniste peuvent être représentées comme des **λ -termes typés** dans le $\lambda\tau$ -calcul.

- Correspondances clés :

Logique intuitionniste	$\lambda\tau$ -calcul typé
Axiome	Variable (var)
Preuve	λ -terme
Proposition	Type
(Ax)	(var)
($\wedge$ I)	(paire)
($\vee$ I1), ($\vee$ I2)	(in1), (in2)
($\Rightarrow$ I)	(abs)
($\wedge$ E1), ($\wedge$ E2)	(π 1), (π 2)
($\Rightarrow$ E)	(app)
($\vee$ E)	(case)

- Exemples de propositions et types :

Proposition logique	$\lambda\tau$ -calcul typé
$\phi \Rightarrow \psi$	$\sigma \rightarrow \tau$
$\phi \wedge \psi$	$\sigma \times \tau$
$\phi \vee \psi$	$\sigma + \tau$

Exemple 1 : $\phi \Rightarrow \psi \Rightarrow \phi$

Typage	Correspondance ($\sigma = \phi$, $\tau = \psi$)
$ \begin{array}{c} \text{(var)} \frac{}{y^\tau, x^\sigma \vdash x : \sigma} \\ \text{(abs)} \frac{}{x^\sigma \vdash \lambda y.x : \tau \rightarrow \sigma} \\ \text{(abs)} \frac{}{\vdash \lambda xy.x : \sigma \rightarrow \tau \rightarrow \sigma} \end{array} $	$ \begin{array}{c} \text{(ax)} \frac{}{\psi, \phi \vdash \phi} \\ (\Rightarrow \text{I}) \frac{}{\phi \vdash \psi \Rightarrow \phi} \\ (\Rightarrow \text{I}) \frac{}{\vdash \phi \Rightarrow \psi \Rightarrow \phi} \end{array} $

Exemple 2 : $\lambda xy. \pi_1(y, x)$

Typage	Correspondance ($\sigma = \phi, \tau = \psi$)
$\text{ge} \quad \frac{\frac{(\text{var}) \text{---}}{y^\tau \vdash y : \tau} \quad \frac{(\text{var}) \text{---}}{x^\sigma \vdash x : \sigma}}{(\text{paire}) \text{---}} \quad \frac{}{x^\sigma, y^\tau \vdash (y, x) : \tau \times \sigma}$	$\frac{\frac{(\text{ax}) \text{---}}{\psi \vdash \psi} \quad \frac{(\text{ax}) \text{---}}{\phi \vdash \phi}}{(\wedge I) \text{---}} \quad \frac{}{\phi, \psi \vdash \psi \wedge \phi}$
$\frac{(\pi_1) \text{---}}{x^\sigma, y^\tau \vdash \pi_1(y, x) : \tau}$	$(\wedge E_1) \text{---} \quad \frac{}{\phi, \psi \vdash \psi}$
$\frac{(\text{abs}) \text{---}}{x^\sigma \vdash \lambda y. \pi_1(y, x) : \tau \rightarrow \tau}$	$(\Rightarrow I) \text{---} \quad \frac{}{\phi \vdash \psi \Rightarrow \psi}$
$\frac{(\text{abs}) \text{---}}{\vdash \lambda xy. \pi_1(y, x) : \sigma \rightarrow \tau \rightarrow \tau}$	$(\Rightarrow I) \text{---} \quad \frac{}{\vdash \phi \Rightarrow \psi \Rightarrow \psi}$

Remarque :

Seules les propositions intuitionnistes prouvables ont un λ -terme typable correspondant, et seuls les λ -termes typables correspondent à des preuves intuitionnistes.

MAGUEMOUN SAMY

